Unity Calculator - Documentation

Overview

`CalculatorManager` is a Unity C# script that manages a **calculator UI and logic**. It handles user input, evaluates expressions following **DMAS rules**, and updates the display.

The script is designed for **maintainability, reusability, and clean input validation**.

Key Features

- Supports basic operations: `+`, , , `÷`
- Follows DMAS (Division → Multiplication → Addition → Subtraction) rules
- Handles decimals and prevents invalid input (like `0.+5` or multiple decimals)
- Maintains a **result state** to handle new input after `=`
- Backspace (`DEL`) and clear (`AC`) functionality
- Result precision: 4 decimal digits
- Singleton pattern for a single instance across scenes

Class Structure

Singleton

```
public static CalculatorManager Instance { get; private set; }
```

- Ensures **only one instance** of the calculator exists
- `DontDestroyOnLoad` keeps it persistent if needed

Methods Overview

Input Handling

`ButtonPressed(string input)`

- Entry point for all button clicks
- Uses a **switch** for command buttons (`AC`, `DEL`, `=`)
- Delegates input validation to helper methods:
  - `HandlePostResultState`
  - `HandleLeadingZero`
  - `HandleOperator`
  - `HandleDecimal`

`HandlePostResultState(string input)`

- Checks if the previous input was a result (after `=`)
- Clears `currentExpression` if a number or decimal is pressed
- Maintains correct calculator behavior for chaining operations

---

`HandleLeadingZero()`

- Prevents numbers like `0666`
- Removes leading `0` if a new digit is pressed

`HandleOperator(string input)`

- Validates operator input:
  - Operators cannot follow another operator
  - Operators cannot follow a decimal
  - can start an expression

`HandleDecimal()`

- Ensures only one decimal per number
- Prepends `0` if a decimal starts a new number (`.5 → 0.5`)

Display & Editing

`UpdateDisplay(string text)`

- Updates the `TextMeshProUGUI` display element

`Backspace()`

- Removes the last character in the expression
- Resets calculator if a result was displayed
- Shows `0` if the expression is empty

`ResetAll()`

- Clears the current expression
- Resets result state
- Sets display to `0`

Expression Parsing & Evaluation

`GetLastNumber()`

- Returns the last number in the current expression
- Used for decimal and leading-zero validation

`IsOperator(string c)`

- Checks if a character is one of the operators (`+`, , , `÷`)

`EvaluateExpression()`

- Evaluates the current expression using DMAS rules
- Uses `Tokenize`, `ProcessMultiplicationDivision`, and `ProcessAdditionSubtraction`
- Formats the result to **4 decimal places**

- Updates the display and sets `isResultDisplayed = true`

`Tokenize(string expression)`

- Converts the expression string into a **list of numbers and operators**
- Handles negative numbers at the start or after another operator

Example:

```
"1+2*3" → ["1", "+", "2", "*", "3"]
```

`ProcessMultiplicationDivision(List<string> tokens)`

- Iterates over tokens and calculates all and `÷` operations **left to right**
- Replaces evaluated tokens with the result

`ProcessAdditionSubtraction(List<string> tokens)`

- Iterates over tokens and calculates remaining `+` and operations **left to right**

Result Formatting

`FormatResult(double value)`

- Rounds the final result to **4 decimal places**
- Removes unnecessary trailing zeros

  Example:

```
3.5000 → 3.5
2.0000 → 2
```

Usage Notes

- Attach the script to an empty GameObject
- Assign a `TextMeshProUGUI` to `displayText`

- Connect buttons to `ButtonPressed(string input)` with their values

Button mapping examples:

- `"AC"` → Clear
- `"DEL"` → Backspace
- `"="` → Evaluate
- `"+"`, `"-"`, `"*"`, `"÷"` → Operators
- `"0".."9"`, `"."` → Numbers
- All input validation is **centralized** and reusable
- Adding new operators (`%`, `^`) only requires:
  1. Update `IsOperator()`
  2. Update evaluation methods
- The Singleton pattern allows other scripts to access the calculator easily:

```
CalculatorManager.Instance.ButtonPressed("5");
```